

Design and Implementation of a Linux Resource Monitor Using Python, Concurrency, and SQLite Persistence

Julio Cesar Blacio Machuca Ariel Jose Llumiquireña Ñacato
Carrera de Ingeniería de Software
Universidad de las Fuerzas Armadas ESPE
Sangolquí, Ecuador

Abstract—Monitoring system resources is essential to understanding the current state of an operating system, yet it can remain an abstract concept when examined only through isolated commands. This paper presents the design and implementation of a Linux terminal application developed in Python 3. The application organizes the six monitoring modules according to the Model–View–Controller pattern: CPU, memory and *swap*, processes, connected users, filesystems, and networking. Data are obtained directly from the */proc* virtual filesystem and, where appropriate, from native commands executed in a controlled manner. To meet operating-systems requirements, the application explicitly creates threads with `threading.Thread` and a child process with `os.fork()`, communicating through a pipe. Complete captures are persisted in SQLite and managed through CRUD operations. Verification performed on Ubuntu under WSL recorded 71 successful tests with no failures or skips in 0.615 s. The result is a traceable educational foundation that integrates observation, concurrency, and persistence without replacing system interfaces with high-level monitoring dependencies.

Index Terms—Linux, resource monitoring, Python, concurrency, SQLite, *proc* filesystem

I. INTRODUCTION

Resource management makes it possible to relate the activity of processes, memory, storage, and communications to the observable behavior of an operating system. In particular, concepts such as processes, scheduling, memory, and input/output require concrete mechanisms that move from theoretical description to verifiable observation [1]. Linux exposes part of that state through kernel interfaces and user-space commands, making it possible to build focused and transparent tools for educational purposes.

The *Linux Resource Monitor* project addresses this need with an interactive console application intended to observe and record the state of a Linux computer without modifying processes, networking, memory, or filesystems. Its scope combines resource collection, an explicit demonstration of threads and processes, and persistence of complete captures. It is not intended to replace specialized tools or to serve as a performance benchmark.

This work contributes a single terminal application that brings together monitoring across six domains, direct */proc* reading, native commands, explicit use of `threading.Thread` and `os.fork()`, and CRUD op-

erations over SQLite. Its structure also provides automated evidence so that functional claims can be reviewed on Linux.

The technical objective is not limited to listing values: it preserves the provenance of each datum from its system source to presentation or storage. This required structured contracts among layers, a partial-failure policy for live queries, and a stricter persistence rule: a capture is stored only when it contains all six required modules. This distinction allows a changing system to be observed without confusing an incomplete live view with a complete historical record.

II. BACKGROUND AND RELATED WORK

The */proc* virtual filesystem publishes information generated by the kernel and running processes; it is therefore a suitable interface for querying counters and descriptors without requiring an intermediate database [2]. This work uses it for CPU, memory, load, and network traffic, while sessions, processes, and mount points are obtained through native utilities that expose these system views.

The two source classes have different properties. Files under */proc* represent state at the instant of each read, whereas utilities produce a snapshot whose composition may change before it is displayed. Consequently, the design does not require a process count to equal a later execution of `ps`; it requires well-formed records with traceable fields. Likewise, cumulative network counters are retained as bytes and packets and are not interpreted as rates without a measurement interval.

From an operating-systems perspective, separating collection, presentation, and coordination prevents terminal formatting from being mixed with resource access. This decision makes it easier to compare measurements, preserve them as captures, and manage errors by module. It also supports the study of the difference between threads that share a process and a child created with `fork()`, without concealing these mechanisms behind high-level monitoring libraries [1].

III. METHODOLOGY

A. Incremental Development

Development was organized over five weeks. The first defined requirements, the MVC architecture, and the relational schema; the second incorporated CPU, memory, and */proc* reading; the third added disk, networking, users, and processes;

the fourth integrated threads, `fork()`, and CRUD; and the fifth was dedicated to integration, tests, manuals, and evidence. This sequence kept each increment associated with a data source and observable acceptance criteria.

B. Capture Flow

A request for the overall state crosses four stages. First, the controller registers one collector function per module. Second, the coordinator creates one `Thread` object for each function, starts all six, and waits for completion. Third, it consolidates three separate structures: `datos`, `errores`, and `evidencias`; the last retains the module name, thread name, and UTC start and finish instants. Finally, the view can present available data and warn about errors. If the user chooses to record the state, the CRUD controller applies an additional validation before delegating the transaction to the repository.

The separation between querying and persistence is deliberate. During a live query, failure of one source does not discard information obtained by the others. By contrast, `consolidar_captura` verifies that CPU, memory, disk, networking, processes, and users are present and that the error dictionary is empty. If a module is missing, a controlled error is raised and no write operation is opened for an incomplete capture.

C. Sources and Calculations

CPU information is described through `/proc/cpuinfo`, `/proc/loadavg`, and two readings of `/proc/stat`. Utilization is not inferred from load average; it is calculated from the deltas of total and idle time,

$$U_{CPU} = 100 \left(1 - \frac{\Delta T_{idle}}{\Delta T_{total}} \right), \quad (1)$$

where both counters come from the aggregate CPU line. For memory, the distinction between free and available memory is preserved: used memory is `MemTotal - MemAvailable`, while used `swap` is `SwapTotal - SwapFree`. Networking uses `/proc/net/dev` for counters and `ip` for addresses; `ps`, `who`, and `df -P -B1` complement the process, user, and mounted-filesystem data.

D. Data Contracts

Models do not return terminal blocks. CPU and memory produce dictionaries; disk, networking, processes, and users produce lists of dictionaries, with one element per mount, interface, process, or session. Stable keys include `porcentaje_uso`, `mem_disponible_mb`, `punto_montaje`, `usuario_propietario`, `direccion_ip`, and `inicio_sesion`. The controller preserves these structures, while the view adds units, rounds values to two decimal places, and builds ASCII tables.

Each parser accepts text and can therefore be tested without querying the current computer. Networking combines counters from `/proc/net/dev` with one address per interface: it prefers the first IPv4 address and uses IPv6 as a fallback; if no address exists, it retains `None`. Disk data are parsed as bytes from `df -P -B1`; persistence converts them to the

GB columns, and historical reads reconstruct bytes before returning the contract to the view. For users, the abbreviated instant reported by `who` is normalized as `YYYY-MM-DD HH:MM`, including year rollover; duration is calculated when `inicio_sesion` is displayed and does not replace the stored value.

E. Concurrency and Safety

Six independent threads, one for each module, are created before the application waits for their completion. Results, errors, and evidence are protected by a lock so that a partial response can be consolidated when a reading fails. The child process starts only after those threads have finished; this order avoids calling `fork()` while other threads are active. The child reads a load metric, communicates a JSON message through a pipe, and is reaped by the parent with `waitpid()`.

The demonstration also checks that no other threads are active before creating the child. Parent and child close the pipe end they do not use; the child terminates with `os._exit()`, and the parent converts the state returned by `waitpid()` into an exit code. A nonzero code or an empty pipe becomes a comprehensible error. The database does not participate in this flow, and no SQLite connection is shared with worker threads or with the child process.

F. Failures and Changing State

Commands are executed with an argument list, captured standard output and error, text mode, a five-second timeout, `LC_ALL=C`, and no `shell=True` [3]. A nonzero return becomes a `RuntimeError`; a missing command, timeout, unavailable `/proc` file, or incomplete content propagates an exception that the worker records under the module name. Thus, the menu receives controlled messages instead of an abrupt termination.

Parsers ignore empty or nonconforming lines when they can continue and reject missing mandatory fields when the result would lose meaning. This criterion matters for tables that change during observation: a process may terminate between two queries, and an interface may have no IP address. The application accepts empty lists and unavailable addresses but does not invent values to complete a capture.

G. Verification Strategy

Unit tests validate parsing functions against sample files so that their results do not depend solely on the current machine. Persistence tests use temporary SQLite databases and cover CRUD operations and cascade deletion. Integrations requiring `fork()` are explicitly limited to Linux. The documented execution was performed on Ubuntu under WSL; therefore, this article does not extend that evidence to other systems.

Verification also covers contracts between layers: the controller preserves partial errors, an incomplete capture is rejected, connections enable foreign keys, update affects only metadata, and deletion removes child rows. Interface tests exercise real dates under a strict format, nonpositive identifiers, explicit confirmation variants, display numbering, deletion

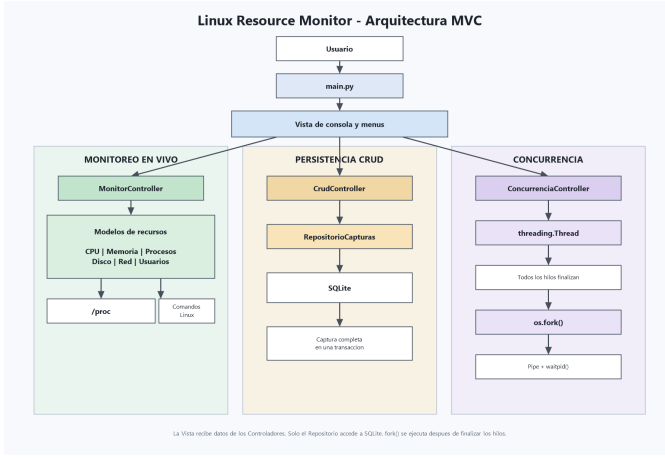


Fig. 1. MVC architecture, concurrent collection, and monitor persistence.

cancellation, and EXITO, ADVERTENCIA, and ERROR messages.

IV. SOLUTION ARCHITECTURE AND DEVELOPMENT

A. MVC Separation

Fig. 1 summarizes the organization. The model reads and transforms system data and encapsulates the SQLite repository. The view requests keyboard input and presents menus, tables, and messages. The controller coordinates collection, validation, concurrency, and the CRUD flow. This separation preserves structured data contracts among layers and prevents SQL queries or `/proc` reads in the view.

The MonitorController facade supports requesting either one module or the overall state without exposing how collection occurs. CrudController receives the concurrent result, requires a complete capture, and translates SQLite failures into an application exception. The repository is the only component that knows SQL statements; consequently, terminal navigation and integrity rules can be tested with doubles without executing Linux commands.

B. Monitoring Modules

Table I relates each module to its primary source and the result delivered to the controller. Command invocations use argument lists, captured output, timeouts, and no `shell=True`; using `subprocess` preserves a clear boundary between the application and system commands [3]. The Python interfaces used for processes and threads correspond to the documented `os` and `threading` APIs [4], [5].

The CPU module separates load from utilization: the three load values come from `/proc/loadavg`, whereas the percentage uses deltas from `/proc/stat`. The count is named logical processors because it comes from `processor` entries in `/proc/cpuinfo`; it is not presented as physical cores. Memory parsing validates five mandatory fields before converting KiB to MiB. Processes, users, and disk each parse one invocation of `ps`, `who`, or `df`, respectively, preserving a snapshot consistent with the invoked source.

TABLE I
TRACEABILITY BETWEEN MODULE, SOURCE, AND COLLECTED DATA.

Module	Primary source	Collected data
CPU	<code>/proc/cpuinfo</code> , <code>/proc/stat</code> , <code>/proc/loadavg</code>	Logical processors, model, frequency, load, and utilization
Memory	<code>/proc/meminfo</code>	Total, free, available, used, and <i>swap</i>
Processes	<code>ps</code>	PID, name, state, and owner
Users	<code>who</code>	Username, terminal, and login start time
Disk	<code>df -P -B1</code>	Mount point, bytes, and utilization percentage
Network	<code>/proc/net/dev</code> , <code>ip</code>	Interface, address, and traffic counters

C. Concurrency and Persistence

Collection starts the six Thread objects, joins them before consolidating the result, and retains errors by module. The child process is created through `os.fork()`, sends its data through a pipe, and is reaped without leaving orphan processes. This sequence makes the difference between thread coordination and interprocess communication explicit.

Captures are stored in a main table with related CPU, memory, disk, process, network, and user metrics. Each connection enables foreign keys, and every complete capture is written in one transaction, so a required failure rolls back the operation. The relationships use cascade deletion to remove associated metrics when a capture is deleted [6], [7]. CRUD supports creating, listing, and consulting captures, updating their metadata, and deleting them through the repository.

The relational design distinguishes cardinalities. CPU and memory have a one-to-one relationship with `capturas`: their primary key is also a foreign key. Disk, processes, networking, and users have one-to-many relationships and their own keys because a capture may include several elements of each type. The parent table retains date, label, comment, and recording user; update changes only the label and comment, never observed metrics.

Writing opens an independent connection and enters the transaction context before inserting the parent row and the six metric families. If a required key is absent, a contract type is invalid, or SQLite rejects a row, the context rolls back the set and the `finally` block closes the connection. During reading, the repository reconstructs the same contract used by the view: one dictionary for CPU and memory and lists for the one-to-many relationships.

D. Terminal Interface Criteria

The main and history menus use numbered options. Invalid input keeps the loop active, and the optional filter requires a real date in YYYY-MM-DD format. Available captures are shown in ascending chronological order before consulting, updating, or deleting; each row has a sequential N. separate from its persistent ID. Existing IDs are not renumbered and, whenever history is empty, any residual sequence is cleared before insertion so the next capture uses ID 1. A valid

TABLE II
VERIFIED RESULTS FROM THE DOCUMENTED EXECUTION.

Validation	Result	Status
Windows suite	71 tests; 2 skips; 0.339 s	0 failures
Ubuntu WSL suite	71 tests; 0 skips; 0.615 s	0 failures
Overall state	6 modules	0 errors
SQLite CRUD	IDs 1–2, order, update, delete	Correct
Concurrency	6 threads + 1 child	Exit status 0

date with no matches is distinguished from an empty history through a specific message. Deletion still requires an explicit `SI`, accepted without case or accent sensitivity. Long listings are initially limited to 20 rows and state how many records are displayed relative to the total.

Output does not depend on color. Percentages are accompanied by `NORMAL`, `ADVERTENCIA`, or `CRITICO`; values include units and two decimal places, and the overall state includes an update timestamp. Missing values are expressed as *No disponible* or with specific empty-list messages. The interface therefore remains understandable in a plain-text terminal and through keyboard operation.

V. RESULTS AND VALIDATION

Table II summarizes the final automated and manual validation. The suites were run on July 22, 2026, with `python -m unittest discover -s tests` on Windows and with `python3` on Ubuntu under WSL. In parallel, a clean clone was exercised directly on Ubuntu through all nine menu options. The documented durations belong to these functional runs and are not presented as performance measurements.

The overall-state option collected all six modules in one query. Panels (a)–(c) of Fig. 2 preserve the three sections of that output: CPU and memory; filesystems and networking; and the process and connected-user lists. The values belong to the observed virtual machine: five assigned logical processors, approximately 4.8 GB of memory, five mounts, two interfaces, and two active sessions. Individual queries from options 2 through 7 remain in the repository evidence index so that each table can be inspected without repeating them in the paper.

The suite combines deterministic tests and integration checks. Parsers for CPU, memory, disk, networking, processes, and users consume fixture files; the repository is created in a temporary directory and verifies both the reconstructed detail and cascade behavior; controllers are exercised with replaceable collectors and repositories; and Linux cases validate the integrated flow and a real `fork()`. This composition reduces dependence on ephemeral values without removing the final check on the target system.

Panels (d)–(e) of Fig. 2 show two consecutive records with IDs 1 and 2. The list places the 15:15:42 capture before the 15:15:52 capture, matching the repository’s ascending chronological order. The second view keeps `N.` and `ID` separate and confirms the updated label for ID 1. Deletion, a filter with no matches, detail consultation, and identifier reset after emptying history were verified during the same walkthrough

and through regression tests; the screenshot containing the previous filter wording remains only in the evidence archive and is not used as a final figure. Panel (f) records six completed threads without errors, distinct parent and child PIDs, and an exit code of zero.

Acceptance criteria were interpreted according to the nature of each datum. Key sets and formulas are required for CPU and memory, command records must be parseable, networking retains one primary address or explicit absence, the live view continues after partial failures, and history writes remain atomic. Exact equality is not required between counts taken at different instants, and observed execution times are not generalized.

VI. CONCLUSIONS

The monitor integrates the functional requirements for observing CPU, memory, processes, users, disk, and networking in a Linux terminal application. MVC separation keeps resource reading, coordination, and presentation distinct; in turn, the repository centralizes SQLite persistence and CRUD operations.

The demonstration of six threads and one child illustrates a relevant ordering decision: threads finish before `fork()` begins, and the parent retrieves the child through a pipe and `waitpid()`. The transaction for every complete capture and cascading foreign keys support the consistency of stored data. The corrected incidents also show the importance of validating contracts among collection, persistence, and presentation. The documented verification supports the behavior described without attributing performance properties to it.

As limitations, the readings depend on the available Linux interfaces and represent changing system states. Future work may study portability in the collection layer or longer-duration samples while preserving the architectural separation and without claiming that these extensions form part of the current version.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. John Wiley & Sons, 2019.
- [2] The Linux Kernel Documentation, “The `/proc` filesystem,” <https://docs.kernel.org/filesystems/proc.html>, accessed: Jul. 17, 2026.
- [3] Python Software Foundation, “subprocess — subprocess management,” <https://docs.python.org/3/library/subprocess.html>, accessed: Jul. 17, 2026.
- [4] Python Software Foundation, “os — miscellaneous operating system interfaces,” <https://docs.python.org/3/library/os.html>, accessed: Jul. 17, 2026.
- [5] Python Software Foundation, “threading — thread-based parallelism,” <https://docs.python.org/3/library/threading.html>, accessed: Jul. 17, 2026.
- [6] SQLite Project, “Sqlite foreign key support,” <https://www.sqlite.org/foreignkeys.html>, accessed: Jul. 17, 2026.
- [7] SQLite Project, “Transaction,” https://www.sqlite.org/lang_transaction.html, accessed: Jul. 17, 2026.

```

Actualizado: 22/07/2026 10:10:23

=== CPU ===
Modelo: 12th Gen Intel(R) Core(TM) i5-12500H
Procesadores lógicos: 5
Frecuencia: 3119.48 Mhz
Carga promedio: 0.90 / 0.00 / 0.02
Uso actual: 0.16 %
Estado: NORMAL

=== MEMORIA ===
Memoria total: 4795.78 MB
Memoria usada: 1586.27 MB
Memoria libre: 1935.19 MB
Memoria d'contorno: 3197.31 MB
Swap total: 0.00 MB
Swap usada: 0.00 MB
Estado: NORMAL

```

(a) CPU and memory.

```

=== DISCO ===
Mostrando 5 de 5 sistemas de archivos

Sistema de archivos | Montaje | Total | Usado | Libre | Uso | Estado |
-----|-----|-----|-----|-----|-----|-----|
tmpfs | /run | 0.47 GB | 0.00 GB | 0.47 GB | 1.00 % | NORMAL |
/dev/sda2 | / | 34.15 GB | 17.15 GB | 15.23 GB | 53.09 % | NORMAL |
tmpfs | /dev/shm | 2.34 GB | 0.00 GB | 2.34 GB | 0.00 % | NORMAL |
tmpfs | /run/lock | 0.09 GB | 0.00 GB | 0.09 GB | 1.00 % | NORMAL |
tmpfs | /run/user/1000 | 0.47 GB | 0.00 GB | 0.47 GB | 1.00 % | NORMAL |

=== RED ===

Interfaz | IP | Bytes recibidos | Bytes enviados | Paquetes recibidos | Paquetes enviados |
-----|-----|-----|-----|-----|-----|
lo | 127.0.0.1 | 42847.00 B | 42847.00 B | 488.00 | 488.00 |
wlp0s3 | 192.168.2.15 | 4535365.00 B | 79338.00 B | 3333.00 | 939.00 |

```

(b) Disk and networking.

```

=== PROCESOS ===
Mostrando 20 de 259 procesos

PID | Nombre | Estado | Usuario |
-----|-----|-----|-----|
1 | systemd | Ss | root |
2 | kthreadd | S | root |
3 | pool_workqueue_release | S | root |
4 | kworker/R-rcu_gp | I< | root |
5 | kworker/R-sync_wq | I< | root |
6 | kworker/R-kvfree_rcu_reclaim | I< | root |
7 | kworker/R-slab_flushwq | I< | root |
8 | kworker/R-netns | I< | root |
11 | kworker/8:8H-events_highpri | I< | root |
12 | kworker/u28:0-ipv6_addrconf | I | root |
13 | kworker/R-mm_percpu_wq | I< | root |
14 | ksoftirqd/0 | S | root |
15 | rcu_preempt | I | root |
16 | rcu_exp_par_gp_kthread_worker/0 | S | root |
17 | rcu_exp_gp_kthread_worker | S | root |
18 | migration/0 | S | root |
19 | idle_inject/0 | S | root |
20 | cpuhp/0 | S | root |
21 | cpuhp/1 | S | root |
22 | idle_inject/1 | S | root |

=== USUARIOS ===

Usuario | Terminal | Inicio de sesion | Duracion |
-----|-----|-----|-----|
jclacio | seat0 | 2026-07-22 09:47 | 0 h 27 min |
jclacio | tty2 | 2026-07-22 09:47 | 0 h 27 min |

```

(c) Processes and connected users.

```

=== HISTORIAL DE CAPTURAS ===
[1] Registrar captura
[2] Listar capturas
[3] Consultar detalle
[4] Actualizar metadatos
[5] Eliminar captura
[0] Volver
Seleccione una opcion: 1
Etiqueta opcional: Primer caso
Comentario opcional:
EXITO: captura 1 registrada.

=== HISTORIAL DE CAPTURAS ===
[1] Registrar captura
[2] Listar capturas
[3] Consultar detalle
[4] Actualizar metadatos
[5] Eliminar captura
[0] Volver
Seleccione una opcion: 1
Etiqueta opcional: Segundo caso
Comentario opcional:
EXITO: captura 2 registrada.

```

(d) Registration of captures 1 and 2.

```

Seleccione una opcion: 2
Fecha YYYY-MM-DD opcional:
N. | ID | FECHA Y HORA | ETIQUETA
1 | 1 | 2026-07-22 15:15:42 | Primer caso
2 | 2 | 2026-07-22 15:15:52 | Segundo caso

=== HISTORIAL DE CAPTURAS ===
[1] Registrar captura
[2] Listar capturas
[3] Consultar detalle
[4] Actualizar metadatos
[5] Eliminar captura
[0] Volver
Seleccione una opcion: 4
N. | ID | FECHA Y HORA | ETIQUETA
1 | 1 | 2026-07-22 15:15:42 | Primer caso
2 | 2 | 2026-07-22 15:15:52 | Segundo caso
Identificador de captura: 1
Nueva etiqueta opcional: Primer caso editado
Nuevo comentario opcional:
EXITO: metadatos actualizados.

```

(e) Listing and update of ID 1.

```

=== MENU PRINCIPAL ===
[1] Estado general
[2] CPU
[3] Memoria y swap
[4] Procesos
[5] Disco
[6] Red
[7] Usuarios conectados
[8] Historial y CRUD
[9] Demostracion de hilos y fork
[0] Salir
Seleccione una opcion: 9
=== DEMOSTRACION DE CONCURRENCIA ===
Hilos completados: 6
Errores de hilos: 0
PID padre: 4065
PID hijo: 4123
Estado de salida: 0
Presione Enter para volver al menu principal:

```

(f) Six threads and one child process.

Fig. 2. Final evidence executed on Ubuntu: overall state (a)–(c), SQLite persistence (d)–(e), and concurrency (f).